CS 260: Data Structures

Week 1

Program Documentation

- 10% of your assignment grades come from comments
- Always identify yourself and the program at the top of the main file

```
/*
<your name>
<date>
<assignment identification>
*/
```

- Always identify yourself and the file contents at the top of other .h and .c files

```
/*
<your name>
<date>
<file description>
*/
```

Comments in Your Code

- Always use descriptive variable/function names
  - int t1 = 1; //BAD - arbitrary, meaningless name
  - int loopcounter = 1; //GOOD - descriptive name
- Provide additional comments as necessary
  - Comment conditional structures
  - Comment function calls
  - Comment variable declaration/assignment
- If you think there is a chance that what you wrote is less than intuitive, then comments are a good idea

Function Documentation

- Always provide a description for each procedure/function.

```
/*
<function description>
<arguments>
<preconditions>
<postconditions>
<return>
*/
```

Precondition / Post-conditions

- The precondition statement indicates what must be true before the function is called
  - The programmer calling the function is responsible for meeting the precondition
  - Example – sqrt function
- The postcondition statement indicates what will be true when the function is finished
  - The programmer writing the function is responsible for ensuring that the postcondition is met
    - Modified params
    - Output is displayed

Function Documentation Example 1

```
/*
    writeSquareRoot : Writes the square root of x to standard output
    Arg1: x - double – the number used for calculations
    Pre: x is greater than or equal to 0
    Post: The square root of x is written to the console
    Return: None
*/

Void writeSquareRoot (double x);
```

- Which meet the precondition?
  - writeSquareRoot (49);
  - writeSquareRoot (1);
  - writeSquareRoot (-9);

Function Documentation Example 2

```
/*
    isVowel: Determines whether x is a vowel or not
    Arg1: x – char – character to be analyzed
    Pre: x – is a character (any case)
    Post: none
    Return: 1 is returned if x is a vowel. return 0 if x is a consonant
*/

Bool isVowel (char x);
```

- Which meet the precondition?
  - isVowel (97);
  - isVowel (‘e’);
  - isVowel (d);

Well-Formatted Output

- Never assume that the person running your program understands anything about the context of the program
- Always label your output and your prompts
- Do not simply display an answer
- Example:
  - #1: X = 7
  - #2: Y = 3
    - Good
  - 7 3
    - Bad
- Always clearly delineate separate output
  - It should be clear where one thing starts and ends to the user

C Overview

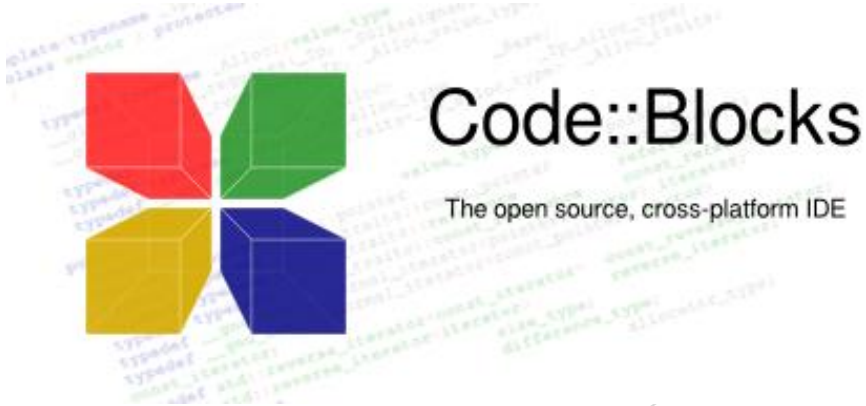
- C is a general-purpose programming language
- Commonly used on computer architectures ranging from the largest supercomputers to the smallest microcomputers and embedded systems
- Created in the 70s by Dennis Ritchie at Bell Labs
- C is a subset of C++
- C-Style languages
  - Curly braces and semicolons
- Not object-oriented
  - Structs

Compiling

- Code will be compiled with the GNU GCC compiler
- Make sure that your code conforms to this standard
  - I will come back to this shortly
  - If code doesn't compile, then I will not be able to run/test it
    - This is going to be a major loss of points
- When compiling, don't expect me to use any special flags
  - No c99
- I will use Code::Blocks IDE
  - Modified version of C90

Code::Blocks

- You are welcome and encouraged to use any IDE
  - Be aware of the compiler you are using
  - Install the GNU GCC compiler with Code::Blocks
- You do not need to use an IDE
  - Working from a command line is a great choice
  - More on this later
  - > gcc hello.c



Miscellaneous C::B Defaults

- More or less c90 standard
  - Not c99 standard
- Boolean data type
  - Don’t exist
- Strings
  - Don’t exist
  - Not as bad as it sounds
- For loop
- Comments
- No classes
  - Use structs
- Many more differences
  - These are the ones that will be most applicable to the course

Comments

- In C90 all we have are block comments
  - /* sgds */
- // is not valid

C Data Types

- Data types
  - Char, int, long, float, void, etc
  - Most *common* data types are present
  - See documentations for details
  - No string or boolean data types
    - We will visit both of these soon

Display Output

- Printf()
  - Used to display data
  - Line breaks
    - Newline character is \n
  - Variables types need to be specified (format specifiers)
    - %i or %d = integer
    - %f = float
    - %c = char
    - %s = string
    - %p = pointer
  - Ex
    - Int age = 21;
    - Char name[] = “Lloyd”;
    - Printf(“My name is %s my age is %d\n”, name, age);

Get Input

- Get input with scanf();
  - Typically used in conjunction with printf()
  - Waits for the user to enter a value
  - Terminated with ENTER
  - Specify the storage location and the type expected
  - Not going to do much of this
- Example
  - Char myName[31];
  - Printf(“Enter your name \n”);
  - Scanf(%s, myName);

Booleans

- True or false
- 1 or 0
- We do not have Booleans in this course
  - Use an int
  - 1 or 0

Arrays

- An array is simply a contiguous segment of memory allocation
- Size is fixed at compile time
- No bells or whistles – just storage space
- Array declaration and assignment examples:

```
Int arr[10] = {34, 23, 10};
Or
Int arr[10];
Arr[0] = 34;
arr[1] = 23;
arr[2] = 10;
```

Strings

- A string is simply an array of chars
  - Ex – char myString[12] = “Hello World”;
  - Memory has been allocated to store 12 chars
  - Memory location symbolically known as *myString*
  - *Strings are terminated by ‘\0’;*
    - Allocate an extra char accordingly
  - Example – Code::Blocks
    - Make the array too small and print
    - Put an escape into a string and print

sizeof

- Function which takes ‘size’ as an argument and returns the number of bytes needed to store it
- Size of data types can vary from system to system
- Crucial for memory allocation
- You will use this a lot

Decision Structures

If and Switch

- IF
  - Standard IF implementation
- Switch statement
  - Standard switch implementation

Example

```
switch(expression) {
    case constant-expression  :
       statement(s);
       break; /* optional */
    case constant-expression  :
       statement(s);
       break; /* optional */
default : /* Optional */
       statement(s);
}
```

Repetition

- Nothing exceptional about use of any of these structures – except for FOR
- While
  - Condition is evaluated at the top of the loop
- Do While
  - Condition is evaluated at the bottom of the loop
- For
  - Declare counter before use

For Loop Example

- Looks good, right?

```
for (int i = 0; i < 10; i++) {
      printf(“%d: \n”, i);
}
```

- This is valid according to c99 standard
  - Need to declare variables before usage

Random Number Generation

- Int Rand()
  - Returns an integer between 0 and RAND_MAX
  - RAND_MAX is a constant dependent on your system – min of 32,767
- How can we use rand() to generate an integer between 1 and 10?
- Rand isn't so random
  - It must be seeded

```
int seed = time(NULL);
srand(seed);
```

  - Requires <time.h>

Print Some Random Numbers

- Use a for loop
- Use rand()

- Seed rand and print again
  - srand(time(NULL))
  - requires time.h header

Function Definitions

```
Data-type function-name(parameters) {
        …;
}
```

Example:

```
Int arraySum(int arr[], int n) {
        int sum = 0;
        int i = 0;
        while(i < n) {
                sum += arr[i];
                i++;
        }
        return sum;
}//function
```

Note that we must pass the array size

sizeOf doesn't work in functions

Structs

- User defined types
  - Can be complex
- Keyword struct
- Structures are similar to classes
  - Only data fields
  - Everything is public
- Ex:

```
struct Student {
        char name[31];
        int age;
};
```

Object Oriented vs Procedural

- In OOP, we define objects and call methods on class instances.
- Ex:

```
Student myStudent = new student();
myStudent.print();
```

- In C, we define functions to use structures, which must be passed to them.
- Ex:

```
Void print(struct student s) {
…
}
Struct student myStudent;
Print(myStudent);
```

Access to Struct Fields

- Dot notation
  - Similar to classes
  - Ex:
    - Student bobcat;
    - bobat.age = 50;
- When working with a pointer to a struct, there is another operator
  - Used to dereference pointers to a structs
  - ->

Typedef

- Useful in naming your own types
- Very handy when used in conjunction with structs
- Capitalized (convention)
- Ex

```
Typedef struct {
        Char name[100];
        Int age;
} Student;
```

- We can have alternate syntax to create students:
  - Student abe
  - struct student abe;

Pointers

- A pointer is a variable whose value is the address of another variable
- Every variable has an address and a value

- Pointer variables are declared with *
- Pointers need types to indicate the type they point to
- Ex
  - Int *ptr
- We can reference the address of a variable with &
- Ex
  - Int x = 7;
  - Int *ptr = &x;
  - The address of x has been assigned to ptr

Dereferencing Pointers

- Dereferencing is getting the value from a pointer variable
- Pointers can be dereferenced with *
  - Used to reference the value instead of the address pointed to

Arrays & Pointers

- Arrays are pointers to the first value in a chunk of memory
  - Ex
  - int arr[10];
  - *arr is the same as arr[0]
  - arr is the same as &arr[0]
- When we pass an array to a function, we are passing the address of the beginning of the array

Pass by Reference vs Value

- Parameters can be passed by reference or by value
- Pass by value creates a copy of the parameter – original value is not modified
  - Most common
- Pass by reference allow the function to alter the values of the parameters directly
  - Efficient
  - Often used with const

Passing by Reference

- Example

```
Void setPi(double *x) {
        *x = 3.14159;
}
```

Argument will be permanently modified

- Function needs to be called with
  - setPi(&someVar);
    - Or you could pass a pointer

Array Parameters are Passed by Reference

- Both do the same thing
  - Void doSomething(int *arr[]);
  - Void soSomething(int arr[])
- Similar to this effect
  - Printf(%p, arr);
  - Printf(%p, &arr[0]);
  - Printf(%p, &arr)
  - All point to the same address

How Large is an Array of Structs?

- A struct is the combined size of the data members
  - Sizeof(struct st)
- An array of structs is the sizeof the struct * the size of the array
  - Sizeof(struct st) * capacity

Dynamic Memory Allocation

- Used to allocate memory
- *Malloc(size)
  - Used to dynamically allocate memory
  - Returns a pointer to the allocated memory
- Ex
  - Char *people; /*create pointer*/
  - people = 0; /*init pointer to null*/
  - people = (char *) malloc(sizeof(char)*10); /*allocate memory and assign pointer to allocated space*/

Dynamic Memory Re-allocation

- *Realloc(*ptr, newSize)
- Used to re-allocate memory
  - Resize
- Example
  - Char *people = malloc(sizeof(char)*10);
  - people = realloc(people, sizeof(char)*20);

Free

- Free()
  - Used to release memory

- Every time you allocate dynamic memory you need to release it
- If you don’t release the memory, then you create a memory leak

Lab

- Practice with pointers, structs, and memory allocation